UNIVERSITATEA DIN BUCUREŞTI
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
DOMENIUL INFORMATICĂ - AN II IF + ID
VARIANTA 2

STUDENT: _____
GRUPA: _____

EXAMEN LA DISCIPLINA "PROGRAMARE AVANSATĂ PE OBIECTE ÎN JAVA"
– SESIUNEA IUNIE 2026 –

NOTĂ:

- Datele de intrare se consideră corecte.
- Nu se vor trata excepţiile şi importurile.
- Punctaj: 5p. (10 x 0.5p.) + 2p. + 2p. + 1p. (din oficiu)

I. Pentru fiecare dintre cele 5 întrebări de mai jos, indicaţi variantele de răspuns pe care le consideraţi corecte:

1. Fie următorul program Java:

```
class Automobil {
    private String marca;
    public Automobil(String marca) { this.marca = marca; }
    public int hashCode() { return marca.length() / 2; }
    public boolean equals(Object obj) {
        return marca.charAt(0) != ((Automobil)obj).marca.charAt(0); }
}
public class Test {
    public static void main(String[] args) {
        HashSet<Automobil> ma = new HashSet<>();
        ma.add(new Automobil("Dacia")); ma.add(new Automobil("Bugatti"));
        ma.add(new Automobil("Ford"));  ma.add(new Automobil("Bentley"));
        ma.add(new Automobil("Fiat"));  ma.add(new Automobil("Delage"));
        System.out.println(ma.size());
    }
}
```

După executarea programului, va fi afişată valoarea:
a) 1 **b)** 2 **c)** 3 **d)** 4

2. Ştiind faptul că şirul de caractere `s` conţine doar paranteze rotunde, deschise sau închise, ce se va afişa pe ecran după executarea secvenţei de mai jos?

```
int n = 0;
int p = s.indexOf("()");
while (p != -1) {
    n += 2;
    s = s.substring(p + 2);
    p = s.indexOf("()");
    if (p != 0) p = -1;
}
System.out.println(n);
```

a) numărul de paranteze din prima grupare consecutivă de perechi `()` din şirul `s`
b) numărul perechi `()` care apar în şirul iniţial `s`, indiferent dacă sunt consecutive sau nu
c) numărul de paranteze rămase în şirul `s` după eliminarea tuturor perechilor corect închise
d) numărul de grupări de paranteze imbricate corect care conţin cel puţin o pereche de paranteze în interiorul altei perechi, de exemplu `(())`, `((()))` ş.a.m.d., din şirul `s`

3. Considerăm următorul program Java:

```
class A { public void print() { System.out.print("A "); } }
class B extends A { }
class C extends B { public void print() { System.out.print("B "); } }

public class Test {
    public static void main(String args[]) {
        A p = new B();
        B q = new C();
        A r = new C();

        p.print();
        q.print();
        r.print();
    }
}
```

După executarea programului, se va afişa:
a) `A B A` **b)** `A B B` **c)** `B B B` **d)** `B B A`

4. Într-o aplicaţie de gestiune urbană considerăm clasele `Cartier`, `Casa`, `Camera` şi `Proprietar`. Specificaţiile funcţionale ale aplicaţiei sunt următoarele:

- `Casa` este compusă din mai multe obiecte de tip `Camera`. Dacă o casă este demolată, atunci toate camerele sale dispar instantaneu.
- Fiecare `Casa` se află într-un `Cartier`. Dacă un `Cartier` este desfiinţat sau reorganizat administrativ, casele nu dispar, ci sunt mutate în mod dinamic în administrarea altui cartier.
- Fiecare `Casa` are un `Proprietar`. Dacă o casă este vândută sau demolată, obiectul `Proprietar` nu trebuie afectat, deoarece acesta poate deţine şi alte proprietăţi în aplicaţie sau poate rămâne în baza de date ca utilizator fără nicio casă.

Care dintre următoarele variante este o abordare corectă de modelare a acestor specificaţii din punct de vedere al programării orientate pe obiecte?

a) Clasa `Casa` extinde clasa `Cartier`, clasa `Camera` extinde clasa `Casa`, iar între clasele `Proprietar` şi `Casa` avem o relaţie de agregare
b) Între clasele `Cartier` şi `Casa` avem o relaţie de agregare, între clasele `Casa` şi `Camera` avem o relaţie de compoziţie, iar între clasele `Casa` şi `Proprietar` avem o relaţie de agregare
c) Între clasele `Casa` şi `Camera` avem o relaţie de agregare, între clasele `Cartier` şi `Casa` avem o relaţie de compoziţie, iar între clasele `Casa` şi `Proprietar` avem o relaţie de agregare
d) Se defineşte şi o clasă abstractă `Oraş` pe care o vor extinde direct toate cele 4 clase precizate anterior

5. Care dintre următoarele afirmaţii referitoare la o clasă abstractă sunt false?
a) constructorul unei clase abstracte `C` nu poate fi apelat direct folosind expresia `C.C()`
b) o clasă abstractă nu poate avea constructori, deoarece nu poate fi instanţiată direct cu operatorul `new`
c) toţi constructorii unei clase abstracte trebuie să fie obligatoriu `private`, pentru a împiedica instanţierea directă a clasei
d) o clasă abstractă poate avea constructori, dar aceştia nu vor fi apelaţi la crearea obiectelor claselor derivate, implicit sau explicit folosind cuvântul cheie `super`

6. Considerăm următorul program Java:

```
public class Test {
    public static void main(String args[]) throws FileNotFoundException {
        Scanner fin = new Scanner(new File("numere.txt"));
        int s = 0;
        try {
            while(fin.hasNext()) {
                int x = fin.nextInt();
                s = s + x;
            }
        }
        catch(ArithmeticException e) {
            System.out.println("Număr incorect!");
        }
        finally {
            System.out.println(s);
        }
    }
}
```

Dacă în fişierul text *numere.txt* există cel puţin două numere scrise incorect (de exemplu: 2a17, a217, abc, 2-17 etc.), atunci programul:

a) va afişa suma numerelor din fişier aflate înaintea primului număr scris incorect, după care programul se va termina forţat din cauza unei excepţii
b) va afişa o singură dată mesajul `Număr incorect!` şi suma numerelor din fişier aflate înaintea primului număr scris incorect, după care programul se va termina normal
c) va afişa câte un mesaj `Număr incorect!` pentru fiecare număr din fişier scris incorect, apoi va afişa suma tuturor numerelor din fişier scrise corect, după care programul se va termina normal
d) va afişa doar mesajul `Număr incorect!`, după care programul se va termina forţat din cauza unei excepţii

7. Considerăm următorul cod Java:

```
class Fir extends Thread {
    int nivel;
    public Fir(int n) { nivel = n; }
    public void run() {
        System.out.print(nivel + " ");
        if (nivel < 3) {
            Fir fir = new Fir(nivel + 1);
            fir.start();
        }
        if (nivel < 1000) {
            Fir fir = new Fir(nivel + 1);
            fir.start();
        }
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        Fir fir = new Fir(0);
        fir.start();
    }
}
```

Presupunând că toate firele create în programul de mai sus reuşesc să se execute complet, care dintre următoarele afirmaţii sunt adevărate?

a) valoarea 3 va fi afişată de 8 ori, deoarece la nivelurile 0, 1 şi 2 numărul firelor se dublează

b) valorile afişate vor apărea obligatoriu în ordine crescătoare, deoarece fiecare fir cu nivelul `n` creează fire doar cu nivelul `n + 1`

c) valoarea 999 va fi afişată de 16 ori

d) valoarea 1000 va fi afişată, deoarece firele cu `nivel = 999` creează fire cu `nivel = 1000`, iar afişarea are loc înaintea verificării condiţiilor din metoda `run()`

8. Considerăm următorul program Java:

```
class Text {
    private String text;
    public Text(String text) { this.text = text; }
    public void schimba(String text) { this.text = text; }
    public void schimba(Text text) { text.schimba("Mihai"); }
    public void copiaza(Text text) { this.text = text.text; }
    public String getText() { return text; }
}
public class Test {
    public static void main(String[] args) {
        Text a = new Text("Alex");
        Text b = new Text("Ion");
        a.schimba("Matei");
        b.schimba(new Text("Dan"));
        a.schimba(b);
        b.copiaza(new Text("Vlad"));
        System.out.println(a.getText() + " " + b.getText());
    }
}
```

După executarea programului, se va afişa:

a) `Matei Vlad` **b)** `Matei Ion` **c)** `Mihai Ion` **d)** `Alex Vlad`

9. Considerăm următorul cod Java:

```
class A {
    int intA;
    public A met1() { return new A(); }
    private void met2() {}
    public void met3() {}
    public int met4(int i) { return 5; }
}
```

```
class B extends A {
    int intB;
    public B met1() { return new B(); }
    public void met2() {}
    private void met3() {}
    public static int met4() { return 5; }
}
```

Care dintre următoarele variante referitoare la codul de mai sus sunt corecte?

a) metoda `met1()` din clasa `B` produce eroare de compilare, deoarece are acelaşi nume şi aceeaşi listă de parametri ca metoda `met1()` din clasa `A`, dar tipurile returnate sunt diferite

b) metoda `met2()` din clasa `B` nu suprascrie metoda `met2()` din clasa `A`, deşi au acelaşi nume şi aceeaşi listă de parametri, deoarece metoda `met2()` din clasa `A` este `private` şi nu este moştenită în clasa `B`

c) metoda `met3()` din clasa `B` produce eroare de compilare, deoarece încearcă să reducă vizibilitatea unei metode `public` moştenite din clasa `A`

d) metoda `met4()` din clasa `B` produce eroare de compilare, deoarece o metodă statică nu poate suprascrie o metodă non-statică cu acelaşi nume din clasa `A`

10. Considerăm următorul program Java:

```
class Test {
    static int x = 10;
    static { x += 6; }
    { x += 3; }
    Test() { x += 2; }

    public static void main(String[] args) {
        Test t1 = new Test();
        Test t2 = new Test();
        System.out.println(x);
    }
}
```

Care dintre următoarele afirmaţii referitoare la o codul de mai sus sunt adevărate?

a) blocul static de iniţializare se execută o singură dată, înainte de apelarea metodei `main()`

b) blocul non-static de iniţializare se execută de fiecare dată când se instanţiază un obiect, după constructorului clasei

c) programul afişează valoarea 26

d) dacă data membră `x` ar fi declarată nestatică, programul ar afişa aceeaşi valoare, deoarece obiectele `t1` şi `t2` sunt create înainte de instrucţiunea de afişare

II. Se consideră definită complet clasa `Concert` având datele membre `denumire`, `localitate`, `data`, `pretBilet` şi `numarLocuri`. Clasa este utilizată pentru a memora informaţii despre concertele propuse de compania `InfoMusic`. Datele membre `denumire`, `localitate` şi `data` sunt de tip `String` (`data` este un şir de forma "`zz-ll-aaaa`"), `numarLocuri` este de tip `int`, iar `pretBilet` este de tip `double`. Clasa încapsulează constructori, metode de tip `set/get` pentru toate datele membre, precum şi metodele `toString()`, `equals()` şi `hashCode()`. Creaţi o listă care să conţină cel puţin 3 obiecte de tip `Concert` şi, folosind stream-uri bazate pe lista creată şi lambda expresii, rezolvaţi următoarele cerinţe:

a) afişaţi lista concertelor din Bucureşti, în ordinea descrescătoare a preţurilor biletelor

b) creaţi o listă care să conţină concertele pentru care numărul de locuri este mai mic decât 500

c) afişaţi topul primelor 3 cele mai ieftine bilete

d) considerând faptul că la toate concertele se vor vinde toate biletele, calculaţi încasările totale estimate ale companiei `InfoMusic` în anul 2026

e) afişaţi, pentru fiecare localitate, concertele care se vor desfăşura în luna iulie

UNIVERSITATEA DIN BUCUREŞTI **STUDENT: _____**
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ **GRUPA: _____**
DOMENIUL INFORMATICĂ - AN II IF + ID
VARIANTA 2

III. Se consideră definită complet o clasă `Angajat` având datele membre `nume`, `vechime` şi `salariu`. Data membră `nume` este de tip `String`, `vechime` este de tip `int`, iar `salariu` este de tip `double`. Clasa încapsulează constructori, metode de tip `set/get` pentru toate datele membre, precum şi metodele `toString()`, `equals()` şi `hashCode()`. Cerinţe:

- ştiind că impozitul unui angajat se calculează doar în funcţie de salariu şi vechime, definiţi o interfaţă funcţională corespunzătoare unei metode pentru calculul impozitului unui angajat;
- definiţi o clasă care să conţină o metodă statică pentru calculul impozitului maxim al angajaţilor memoraţi într-o listă cu elemente de tip `Angajat` folosind o formulă de calcul generică, transmisă utilizând mecanismul de callback;
- folosind o lambda expresie, exemplificaţi utilizarea clasei utilitare pentru o lista cu elemente de tip `Angajat` şi următoarea formulă de calcul a impozitului: `impozit = salariu/100 + vechime*10`.

UNIVERSITATEA DIN BUCUREŞTI
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
DOMENIUL INFORMATICĂ - AN II IF + ID
VARIANTA 2

STUDENT: _____
GRUPA: _____

UNIVERSITATEA DIN BUCUREŞTI
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
DOMENIUL INFORMATICĂ - AN II IF + ID
VARIANTA 2

STUDENT: _____
GRUPA: _____

EXAMEN LA DISCIPLINA "PROGRAMARE AVANSATĂ PE OBIECTE ÎN JAVA"
– SESIUNEA IUNIE 2026 –

NOTĂ:

- Datele de intrare se consideră corecte.
- Nu se vor trata excepţiile şi importurile.
- Punctaj: 5p. (10 x 0.5p.) + 2p. + 2p. + 1p. (din oficiu)

I. Pentru fiecare dintre cele 5 întrebări de mai jos, indicaţi variantele de răspuns pe care le consideraţi corecte:

1. Fie următorul program Java:

```
class Automobil {
    private String marca;
    public Automobil(String marca) { this.marca = marca; }
    public int hashCode() { return marca.length() / 2; }
    public boolean equals(Object obj) {
        return marca.charAt(0) != ((Automobil)obj).marca.charAt(0); }
}
public class Test {
    public static void main(String[] args) {
        HashSet<Automobil> ma = new HashSet<>();
        ma.add(new Automobil("Dacia")); ma.add(new Automobil("Bugatti"));
        ma.add(new Automobil("Ford"));  ma.add(new Automobil("Bentley"));
        ma.add(new Automobil("Fiat"));  ma.add(new Automobil("Delage"));
        System.out.println(ma.size());
    }
}
```

După executarea programului, va fi afişată valoarea:
a) 4 **b)** 3 **c)** 2 **d)** 1

2. Ştiind faptul că şirul de caractere `s` conţine doar paranteze rotunde, deschise sau închise, ce se va afişa pe ecran după executarea secvenţei de mai jos?

```
int n = 0;
int p = s.indexOf("()");
while (p != -1) {
    n += 2;
    s = s.substring(p + 2);
    p = s.indexOf("()");
    if (p != 0) p = -1;
}
System.out.println(n);
```

a) numărul perechi `()` care apar în şirul iniţial `s`, indiferent dacă sunt consecutive sau nu
b) numărul de paranteze din prima grupare consecutivă de perechi `()` din şirul `s`
c) numărul de paranteze rămase în şirul `s` după eliminarea tuturor perechilor corect închise
d) numărul de grupări de paranteze imbricate corect care conţin cel puţin o pereche de paranteze în interiorul altei perechi, de exemplu `(())`, `((()))` ş.a.m.d., din şirul `s`

3. Considerăm următorul program Java:

```
class A { public void print() { System.out.print("A "); } }
class B extends A { }
class C extends B { public void print() { System.out.print("B "); } }

public class Test {
    public static void main(String args[]) {
        A p = new B();
        B q = new C();
        A r = new C();

        p.print();
        q.print();
        r.print();
    }
}
```

Se punctează maxim oricare dintre variantele de răspuns următoare: doar b), doar c) sau b) şi c)!
După executarea programului, se va afişa:
a) `A B A` **b)** `A B B` **c)** `A B B` **d)** `B B A`

4. Într-o aplicaţie de gestiune urbană considerăm clasele `Cartier`, `Casa`, `Camera` şi `Proprietar`. Specificaţiile funcţionale ale aplicaţiei sunt următoarele:

- `Casa` este compusă din mai multe obiecte de tip `Camera`. Dacă o casă este demolată, atunci toate camerele sale dispar instantaneu.
- Fiecare `Casa` se află într-un `Cartier`. Dacă un `Cartier` este desfiinţat sau reorganizat administrativ, casele nu dispar, ci sunt mutate în mod dinamic în administrarea altui cartier.
- Fiecare `Casa` are un `Proprietar`. Dacă o casă este vândută sau demolată, obiectul `Proprietar` nu trebuie afectat, deoarece acesta poate deţine şi alte proprietăţi în aplicaţie sau poate rămâne în baza de date ca utilizator fără nicio casă.

Care dintre următoarele variante este o abordare corectă de modelare a acestor specificaţii din punct de vedere al programării orientate pe obiecte?

a) Clasa `Casa` extinde clasa `Cartier`, clasa `Camera` extinde clasa `Casa`, iar între clasele `Proprietar` şi `Casa` avem o relaţie de agregare
b) Se defineşte şi o clasă abstractă `Oraş` pe care o vor extinde direct toate cele 4 clase precizate anterior
c) Între clasele `Cartier` şi `Casa` avem o relaţie de agregare, între clasele `Casa` şi `Camera` avem o relaţie de compoziţie, iar între clasele `Casa` şi `Proprietar` avem o relaţie de agregare
d) Între clasele `Casa` şi `Camera` avem o relaţie de agregare, între clasele `Cartier` şi `Casa` avem o relaţie de compoziţie, iar între clasele `Casa` şi `Proprietar` avem o relaţie de agregare

5. Care dintre următoarele afirmaţii referitoare la o clasă abstractă sunt false?
a) o clasă abstractă nu poate avea constructori, deoarece nu poate fi instanţiată direct cu operatorul `new`
b) constructorul unei clase abstracte `C` nu poate fi apelat direct folosind expresia `C.C()`
c) toţi constructorii unei clase abstracte trebuie să fie obligatoriu `private`, pentru a împiedica instanţierea directă a clasei
d) o clasă abstractă poate avea constructori, dar aceştia nu vor fi apelaţi la crearea obiectelor claselor derivate, implicit sau explicit folosind cuvântul cheie `super`

6. Considerăm următorul program Java:

```
public class Test {
    public static void main(String args[]) throws FileNotFoundException {
        Scanner fin = new Scanner(new File("numere.txt"));
        int s = 0;
        try {
            while(fin.hasNext()) {
                int x = fin.nextInt();
                s = s + x;
            }
        }
        catch(ArithmeticException e) {
            System.out.println("Număr incorect!");
        }
        finally {
            System.out.println(s);
        }
    }
}
```

Dacă în fişierul text *numere.txt* există cel puţin două numere scrise incorect (de exemplu: 2a17, a217, abc, 2-17 etc.), atunci programul:

a) va afişa o singură dată mesajul `Număr incorect!` şi suma numerelor din fişier aflate înaintea primului număr scris incorect, după care programul se va termina normal
b) va afişa câte un mesaj `Număr incorect!` pentru fiecare număr din fişier scris incorect, apoi va afişa suma tuturor numerelor din fişier scrise corect, după care programul se va termina normal
c) va afişa doar mesajul `Număr incorect!`, după care programul se va termina forţat din cauza unei excepţii
d) va afişa suma numerelor din fişier aflate înaintea primului număr scris incorect, după care programul se va termina forţat din cauza unei excepţii

7. Considerăm următorul cod Java:

```
class Fir extends Thread {
    int nivel;
    public Fir(int n) { nivel = n; }
    public void run() {
        System.out.print(nivel + " ");
        if (nivel < 3) {
            Fir fir = new Fir(nivel + 1);
            fir.start();
        }
        if (nivel < 1000) {
            Fir fir = new Fir(nivel + 1);
            fir.start();
        }
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        Fir fir = new Fir(0);
        fir.start();
    }
}
```

Presupunând că toate firele create în programul de mai sus reuşesc să se execute complet, care dintre următoarele afirmaţii sunt adevărate?

a) valorile afişate vor apărea obligatoriu în ordine crescătoare, deoarece fiecare fir cu nivelul `n` creează fire doar cu nivelul `n + 1`

b) valoarea 3 va fi afişată de 8 ori, deoarece la nivelurile 0, 1 şi 2 numărul firelor se dublează

c) valoarea 999 va fi afişată de 16 ori

d) valoarea 1000 va fi afişată, deoarece firele cu `nivel = 999` creează fire cu `nivel = 1000`, iar afişarea are loc înaintea verificării condiţiilor din metoda `run()`

8. Considerăm următorul program Java:

```
class Text {
    private String text;
    public Text(String text) { this.text = text; }
    public void schimba(String text) { this.text = text; }
    public void schimba(Text text) { text.schimba("Mihai"); }
    public void copiaza(Text text) { this.text = text.text; }
    public String getText() { return text; }
}
public class Test {
    public static void main(String[] args) {
        Text a = new Text("Alex");
        Text b = new Text("Ion");
        a.schimba("Matei");
        b.schimba(new Text("Dan"));
        a.schimba(b);
        b.copiaza(new Text("Vlad"));
        System.out.println(a.getText() + " " + b.getText());
    }
}
```

După executarea programului, se va afişa:

a) `Alex Vlad` **b)** `Mihai Ion` **c)** `Matei Ion` **d)** `Matei Vlad`

9. Considerăm următorul cod Java:

```
class A {
    int intA;
    public A met1() { return new A(); }
    private void met2() {}
    public void met3() {}
    public int met4(int i) { return 5; }
}
```

```
class B extends A {
    int intB;
    public B met1() { return new B(); }
    public void met2() {}
    private void met3() {}
    public static int met4() { return 5; }
}
```

Care dintre următoarele variante referitoare la codul de mai sus sunt corecte?

a) metoda `met1()` din clasa `B` produce eroare de compilare, deoarece are acelaşi nume şi aceeaşi listă de parametri ca metoda `met1()` din clasa `A`, dar tipurile returnate sunt diferite

b) metoda `met2()` din clasa `B` nu suprascrie metoda `met2()` din clasa `A`, deşi au acelaşi nume şi aceeaşi listă de parametri, deoarece metoda `met2()` din clasa `A` este `private` şi nu este moştenită în clasa `B`

c) metoda `met3()` din clasa `B` produce eroare de compilare, deoarece încearcă să reducă vizibilitatea unei metode `public` moştenite din clasa `A`

d) metoda `met4()` din clasa `B` produce eroare de compilare, deoarece o metodă statică nu poate suprascrie o metodă non-statică cu acelaşi nume din clasa `A`

10. Considerăm următorul program Java:

```
class Test {
    static int x = 10;
    static { x += 6; }
    { x += 3; }
    Test() { x += 2; }

    public static void main(String[] args) {
        Test t1 = new Test();
        Test t2 = new Test();
        System.out.println(x);
    }
}
```

Care dintre următoarele afirmaţii referitoare la o codul de mai sus sunt adevărate?

a) blocul non-static de iniţializare se execută de fiecare dată când se instanţiază un obiect, după constructorului clasei

b) programul afişează valoarea 26

c) dacă data membră `x` ar fi declarată nestatică, programul ar afişa aceeaşi valoare, deoarece obiectele `t1` şi `t2` sunt create înainte de instrucţiunea de afişare

d) blocul static de iniţializare se execută o singură dată, înainte de apelarea metodei `main()`

II. Se consideră definită complet clasa `Concert` având datele membre `denumire`, `localitate`, `data`, `pretBilet` şi `numarLocuri`. Clasa este utilizată pentru a memora informaţii despre concertele propuse de compania `InfoMusic`. Datele membre `denumire`, `localitate` şi `data` sunt de tip `String` (`data` este un şir de forma "`zz-ll-aaaa`"), `numarLocuri` este de tip `int`, iar `pretBilet` este de tip `double`. Clasa încapsulează constructori, metode de tip `set/get` pentru toate datele membre, precum şi metodele `toString()`, `equals()` şi `hashCode()`. Creaţi o listă care să conţină cel puţin 3 obiecte de tip `Concert` şi, folosind stream-uri bazate pe lista creată şi lambda expresii, rezolvaţi următoarele cerinţe:

a) afişaţi lista concertelor din Bucureşti, în ordinea crescătoare a preţurilor biletelor

b) creaţi o listă care să conţină concertele pentru care numărul de locuri este mai mare decât 500

c) afişaţi topul primelor 3 cele mai scumpe bilete

d) considerând faptul că la toate concertele se vor vinde toate biletele, calculaţi încasările totale estimate ale companiei `InfoMusic` în anul 2026

e) afişaţi, pentru fiecare luna, concertele care se vor desfăşura în luna respectivă

III. Se consideră definită complet o clasă `Produs` având datele membre `denumire`, `cantitate` şi `preț`. Data membră `denumire` este de tip `String`, `cantitate` este de tip `int`, iar `pret` este de tip `double`. Clasa încapsulează constructori, metode de tip `set/get` pentru toate datele membre, precum şi metodele `toString()`, `equals()` şi `hashCode()`. Cerințe:

- ştiind că valoarea reducerii unui produs se calculează doar în funcţie de `preț` şi `cantitate`, definiţi o interfaţă funcţională corespunzătoare unei metode pentru calculul reducerii unui produs;
- definiţi o clasă care să conţină o metodă statică pentru calculul reducerii minime a produselor memorate într-o listă cu elemente de tip `Produs`, folosind o formulă de calcul generică, transmisă utilizând mecanismul de callback;
- folosind o lambda expresie, exemplificaţi utilizarea clasei utilitare pentru o listă cu elemente de tip `Produs` şi următoarea formulă de calcul a reducerii: `reducere = pret * cantitate / 10`.

UNIVERSITATEA DIN BUCUREŞTI
FACULTATEA DE MATEMATICĂ ŞI INFORMATICĂ
DOMENIUL INFORMATICĂ - AN II IF + ID
VARIANTA 2

STUDENT: _____
GRUPA: _____